

TDV 2215 Terminal Emulator ROM Analysis

Executive Summary

The TDV 2215 is a sophisticated Z80-based terminal emulator implementing VT100-like functionality with extensions. Based on Ghidra analysis, the firmware demonstrates a well-structured state machine for terminal emulation with comprehensive I/O handling, NVRAM configuration storage, and extensive character processing capabilities.

System Architecture Overview

Memory Map

- **RAM:** `0x0000-0x1FFF` (8KB system RAM)
- **ROM:** `0x2000-0x3FFF` (8KB EPROM - NMOS 2716)
- **Static RAM:** `0x4000-0x47FF` (2KB buffer/workspace)
- **CNVD RAM:** `0x5800-0x5FFF` (2KB display memory)
- **Display Memory:** `0x6000-0x7FFF` (8KB video buffer)
- **I/O Space:** `0x00-0xFE`

System Entry Point

Address: `0x0000` (RST 0 vector) **Function:** `RST0()`

Entry sequence:

1. Disable maskable interrupts (`DI`)
2. Jump to `terminal_boot_sequence()` at `0x01E4`

Boot Sequence and Initialization

Primary Boot Function: `terminal_boot_sequence()` (**0x01E4**)

c

```
void terminal_boot_sequence(void) {
    disableMaskableInterrupts();

    // Hardware stabilization delay (0x3000 Loops)
    sVar2 = 0x3000;
    do {
        sVar2 = sVar2 + -1;
    } while ((char)sVar2 != '\0' || (char)((ushort)sVar2 >> 8) != '\0');

    // Check for ROM at 0x8000 (extension ROM)
    if (((byte)(DAT_ram_8000 + DAT_ram_8001) == '\0') && (CARRY1(DAT_ram_8000,DAT_ram_8001))) {
        _boot_progress_marker = 0x234;
        initialize_system_variables();
        halt_baddata(); // System halt on ROM check failure
    }

    // Cold vs warm boot detection
    bVar1 = (byte)((ushort)_system_status_word >> 8);
    cVar3 = !CARRY1(bVar1,(byte)_system_status_word);

    if (((byte)(bVar1 + (byte)_system_status_word) != '\0') || ((bool)cVar3)) {
        _boot_progress_marker = 0x24a;
        hardware_power_on_test(); // Power-on self-test
    }

    // Initialize system variables and hardware
    _boot_progress_marker = 0x24d;
    initialize_system_variables();

    // Display startup message and error codes if needed
    if (cVar3 != '\0') {
        write_string(); // Display startup banner
        display_error_code_custom_chars();
        write_to_keyboard_return_carry_if_busy(0x18);
    }

    _system_status_word = 0x8080; // Set system ready
    call_hardware_extension(); // Initialize hardware extensions
    write_string(); // Final startup message

    // Infinite Loop - should never reach here
    do {
    } while(true);
}
```

System Variable Initialization: `initialize_system_variables()` (0x07FD)

Critical Setup Operations:

1. Interrupt Configuration:

- `_ISR_RETURN_ADDRESS = 0x10`
- `ISR_65_RETURN_ADDRESS = 0x10`
- `ISR_VSCYN_RETURN_ADDRESS = 0x10`

2. Hardware Ports:

- `ContolPort1_value = 0x48`
- `value_for_statusport_2 = 0xf2`

3. Display Setup:

- `cursor_row_position = 0x18`
- `_current_cursor_character_address = 0x6000`
- `_next_cursor_character_address = 0x101`

4. Keyboard Timeout:

- `keyboard_activity_timeout_counter = 30000`

5. Function Pointer Initialization:

- All interrupt handlers initially set to `0x29e`
- Configuration template pointer set to `&config_template_table`

Main Execution Loop

Primary Loop: `terminal_main_loop()` (0x1FCE)

The main loop initializes the terminal state machine and sets up the character processing pipeline:

c

```
void terminal_main_loop(void) {
    // Initialize terminal buffers
    check_extension_and_start_config_menu(1);

    // Clear terminal buffer
    pbVar2 = &terminal_buffer_start;
    do {
        *pbVar2 = 0;
        pbVar2 = pbVar2 + 1;
        compare_de_hl_registers();
    } while (in_CY_flag == '\0');

    // Set up interrupt handlers
    _saved_vscyn_isr_address = ISR_VSCYN_RETURN_ADDRESS;
    ISR_VSCYN_RETURN_ADDRESS = 0x228d;
    _saved_dispatch_function_ptr = dispatch_function_pointer_2;
    _default_handler_address = 0x20c9;

    // Initialize display and configuration
    create_bit_pattern_display();
    _system_config_pointer = &UNK_ram_3921;
    _display_function_pointer = send_form_feed_sequence;
    _config_template_pointer = &more_config_data;
    load_config_from_nvram();

    // Set up keyboard and communication handlers
    sw_printer_mode = 1;
    keyboard_handler_function_pointer = 0x2239;
    configure_display_baud_settings();
    initialize_display_registers();
    initialize_terminal_display_mode();

    // Set main loop function pointer and call
    _main_loop_function_pointer = (code *)0x20cd;
    (*_main_loop_function_pointer)(); // Jump to character processing loop
}
```

Character Processing Loop: special_character_processor() (0x20CD)

Function: Processes input characters and routes them through the state machine.

```

c
void special_character_processor(byte param_1) {
    if (0xaf < param_1) {
        high_character_dispatch_table(); // Handle high characters (0xB0+)
        return;
    }

    _input_handler_address = &DAT_ram_581c;
    load_push_key_string_from_nvram(param_1 & 0xf); // PUSH-key handling
    (*_main_loop_function_pointer)(); // Continue processing
}

```

Terminal State Machine

Primary Character Dispatcher

The terminal uses a dual-dispatch system for character processing:

1. Control Character Dispatcher: `control_character_dispatcher()` (0x25A0)

- **Range:** 0x00-0x1F (control characters)
- **Method:** Jump table lookup using $(\text{param_1} \& 0x1f) * 2$ as offset
- **Table Location:** Address in HL register + calculated offset

2. High Character Dispatcher: `high_character_dispatch_table()` (0x25A0)

- **Range:** 0xB0+ (extended characters)
- **Method:** Jump table lookup using $(\text{param_1} \& 0xf) * 2$ as offset
- **Purpose:** Handles semigraphic characters (line drawing, histogram, subscript/superscript)

Escape Sequence Processing

Escape Sequence Parser: `escape_sequence_parser()` (0x22FB)

Function: Processes ESC sequences per ECMA-6/35/48 standards.

Operating Modes:

1. **TDV 2115 Mode** (default): Ignores ESC except `ESC Q` (enables Extended Control)
2. **Extended Mode:** Handles:
 - C1 controls (ESC 40-5F)
 - CSI sequences (ESC I)
 - Three-character sequences (DWL/SWL)

- Device control strings (DCS for PUSH-key loading)

Character Output Processing

Terminal Character Output: `terminal_character_output()` (0x2557)

Purpose: Handles display of 256-character set including:

- ASCII (20-7F)
- Control codes (00-1F)
- Semigraphic characters (line drawing, histogram, subscript, superscript, plot)

Processing Flow:

1. Store character in `current_output_character`
2. Check cursor wrap: `cursor_wrap_enabled_flag`
3. Process through escape sequence parser
4. Handle serial transmission if needed
5. Update display

Interrupt System

VSYNC Interrupt Handler: `vsync_interrupt_handler()` (0x052C)

Primary Functions:

1. Display Refresh Control:

- Sets `set_to_1_by_vsync = 1`
- Manages display refresh timing via `display_refresh_control`
- Handles video-off control: `video_off_control_flag`

2. Keyboard Activity Timeout:

- Decrements `keyboard_activity_timeout_counter`
- Timeout value: 30000 cycles
- Manages `interrupt_activity_flag`

3. Timer Channel Management:

- Processes `timer_channel_flags` (8 channels)
- Handles printer timer: `_printer_timer_counter`
- Calls `check_printer_status()` when needed

4. Status Port Updates:

- Updates `value_for_statusport_2`

- Manages hardware status indicators

Keyboard Interrupt Handler: `keyboard_buffer_interrupt_handler()` (0x0443)

Functions:

- Sets `interrupt_activity_flag = 0x5e`
- Processes input from detachable keyboard
- Handles convenience switches:
 - Key Click (4.1.2)
 - Auto Repeat (4.1.4)
 - Key Rollover (4.1.5)
 - CAPS on Power Up (4.1.6)
- Manages PUSH-key sequence transmission

Serial Communication Handler: `host_serial_interrupt_handler()` (0x0295)

Functions:

- Sets `interrupt_activity_flag = 0x5e`
- Handles Z80 SIO communication
- Manages XON/XOFF handshake
- Supports multiple interfaces: V.24 (RS-232-C), V.11 (RS-422), 20mA current loop

Display Memory Management

Cursor Management

Cursor Position Functions:

- `setup_cursor_position()` (0x0064): Initialize cursor position
- `advance_cursor_one_character()` (0x0073): Move cursor forward
- `move_cursor_back_one_position()` (0x007F): Move cursor backward
- `update_cursor_memory_addresses()` (0x0091): Update memory pointers

Cursor Movement Handler: `cursor_movement_handler()` (0x31E9)

Function: Processes cursor positioning commands and updates display memory addresses.

Display Operations

Line Operations:

- `clear_display_line()` (0x0103): Clear current line
- `scroll_display_region_up()` (0x17E6): Scroll region upward
- `scroll_display_region_down()` (0x1778): Scroll region downward
- `insert_line_at_cursor()` (0x011B): Insert blank line
- `delete_line_at_cursor()` (0x182A): Remove line

Character Operations:

- `write_char_to_video_memory()` (0x00A6): Write character to display
- `write_char_to_display_with_timing()` (0x009A): Timed character write
- `display_string_until_control_char()` (0x00A9): String output

Configuration System

NVRAM Configuration Storage

NVRAM Memory Map (ER3400 Non-Volatile RAM):

Soft-Switch Configuration (0x50-0x5A, 11 bytes):

- 37 soft-switches packed in bit fields
- Categories: Convenience, Function, Communication
- Extracted via template table at 0x0AA0-0x0AAA into RAM at 0x5F00-0x5F24

PUSH-Key Storage (0x60-0x17B):

- 16 sequences total (8 keys × 2 shifts)
- 12 bytes maximum per sequence
- **Layout:**
 - PUSH-Key 1: 0x60-0x6B (normal), 0x6C-0x77 (shifted)
 - PUSH-Key 2: 0x78-0x83 (normal), 0x84-0x8F (shifted)
 - PUSH-Key 3: 0x90-0x9B (normal), 0x9C-0xA7 (shifted)
 - PUSH-Key 4: 0xA8-0xB3 (normal), 0xB4-0xBF (shifted)
 - PUSH-Key 5: 0xC0-0xCB (normal), 0xCC-0xD7 (shifted)
 - PUSH-Key 6: 0xD8-0xE3 (normal), 0xE0-0xEB (shifted)
 - PUSH-Key 7: 0x100-0x10B (normal), 0x120-0x12B (shifted)
 - PUSH-Key 8: 0x140-0x14B (normal), 0x170-0x17B (shifted)

Configuration Functions:

- `load_config_from_nvram()` (0x015A): Load settings from NVRAM
- `save_config_to_nvram()` (0x015D): Save settings to NVRAM
- `factory_reset_nvram()` (0x3820): Reset to factory defaults

Hardware I/O Interface

Port Assignments:

- **Status Port:** Hardware status monitoring
- **Control Port 1:** `ContolPort1_value = 0x48`
- **Control Port 2:** Display attribute control
- **Display Attribute Port:** Character attribute management
- **Keyboard Data Port:** Key input processing
- **Serial Ports:** Z80 SIO channels A & B

Hardware Test Functions:

- `hardware_power_on_test()` (0x0AAB): Comprehensive hardware validation
- `test_z80sio_ports()` (0x0C38): Serial interface testing
- `test_keyboard_communication()` (0x0CF5): Keyboard interface testing
- `nvram_range_memory_test()` (0x0B8E): NVRAM validation

State Variables and Flags

Critical State Variables:

Display State:

- `cursor_row_position`: Current cursor row (0-23)
- `current_cursor_column`: Current cursor column (0-79)
- `_current_cursor_character_address`: Memory address of cursor position
- `cursor_wrap_enabled_flag`: Enable/disable cursor wrap
- `video_off_control_flag`: Display on/off control

System State:

- `_system_status_word`: Overall system status (0x8080 = ready)
- `serial_transmission_busy_flag`: Serial transmission state
- `interrupt_activity_flag`: Interrupt processing indicator
- `keyboard_activity_timeout_counter`: Keyboard timeout (30000 cycles)

Configuration State:

- `sw_printer_mode`: Printer mode setting
- `sw_roll_page_mode`: Roll/page mode selection
- `keyboard_transmission_enable`: Keyboard transmission control

Error Handling:

- `error_message_flag`: Error display control
- `nvramp_reset_flag`: NVRAM reset indicator
- `_boot_progress_marker`: Boot sequence progress tracking

Terminal Features and Capabilities

Character Set Support:

- **ASCII Set**: Standard 20-7F characters
- **Semigraphic Set**: Line drawing, histogram, subscript, superscript, plot characters
- **Total**: 256 characters with upper/lowercase support

Display Features:

- **Screen Size**: 15" with anti-reflex faceplate
- **Character Attributes**: Normal, inverse, underline, blink combinations
- **Double-Width Characters**: Supported via DWL sequences
- **Graphic Rendition**: ATTR, UNDERLINE, and SGR modes

Communication Features:

- **Baud Rates**: 50, 75, 110, 134.5, 200, 300, 600, 1200, 2400, 4800, 9600, 19200
- **Interfaces**: V.24 (RS-232-C), V.11 (RS-422), 20mA current loop
- **Handshake**: XON/XOFF flow control
- **Mode**: Character-by-character with simultaneous send/receive

Advanced Features:

- **Transparent Mode**: For debugging
- **Device Status Report**: ESC [5 n and ESC [6 n
- **Direct Cursor Addressing**: ESC [Pn ; Pn H
- **Insert/Delete Operations**: With numeric parameters
- **Print Buffer**: Optional 2000-character capacity

Known Issues and Anomalies

Identified Bugs:

1. Infinite Loop Bug: `clear_memory_range_infinite_loop_bug()` (0x0A58)

Location: Called from `initialize_system_variables()` **Issue:** Function name suggests potential infinite loop condition **Impact:** Could cause system hang during initialization

2. Jump Table Recovery Issues:

Locations:

- `control_character_dispatcher()` (0x25A0)
- `high_character_dispatch_table()` (0x25A0) **Issue:** Ghidra unable to recover jump tables due to complexity **Impact:** Exact control character mapping unclear

3. Bad Instruction Data:

Location: `terminal_boot_sequence()` **Issue:** Warning about bad instruction data causing control flow truncation **Impact:** Potential code path unreachability

Vestigial Code Paths:

1. Hardware Extension Calls:

Function: `call_hardware_extension()` (0x027C) **Usage:** Called multiple times during boot **Observation:** May be placeholder for optional hardware modules

2. ROM Extension Check:

Location: Boot sequence checks for ROM at 0x8000 **Purpose:** Suggests support for expansion ROM **Current State:** Appears unused in standard configuration

Development Recommendations

For Emulation Implementation:

- Focus on State Machine:** The dual-dispatch character processing system is the core of terminal emulation
- Interrupt Timing:** VSYNC interrupt handler is critical for proper display timing
- NVRAM Emulation:** Configuration persistence requires accurate NVRAM modeling
- Character Set:** Implement full 256-character support including semigraphics
- Jump Table Analysis:** Manual analysis needed for complete control character mapping

For Debugging:

1. **Monitor State Variables:** Track `_system_status_word`, `interrupt_activity_flag`, and cursor position
2. **Boot Sequence:** Watch `_boot_progress_marker` for initialization problems
3. **Character Flow:** Trace from keyboard input through `special_character_processor()` to display output
4. **Error Codes:** Implement `display_error_code_custom_chars()` for diagnostics

For Reverse Engineering:

1. **Complete Jump Tables:** Manual disassembly needed at 0x25B3 region
 2. **Escape Sequence Mapping:** Full ESC sequence support requires detailed analysis
 3. **Hardware Interface:** I/O port mapping needs hardware documentation correlation
 4. **Timer System:** Channel timing and printer interface require further investigation
-

Analysis completed using Ghidra decompilation of TDV 2215 ROM dump

Target Platform: Z80-based terminal system

Firmware Version: Revision Level 11, February 1983

Manufacturer: TANDBERG DATA A/S, Part no. 385604